Tarea 1: Cálculo y verificación de hashes

Apartado 1. Cálculo de hashes

Algoritmo	Longitud
MD5	32 caracteres
SHA-1	40 caracteres
SHA-256	64 caracteres
SHA-512	128 caracteres
`b2sum`	128 caracteres

Cambio de un carácter

Hash original:

```
openssl dgst -sha256 mensaje.txt
SHA2-256(mensaje.txt)= 3b3c1a1d6319561bd1ebf8c214cd77833e6
```

Después de cambiar el carácter `s` de «hamburguesa»:

```
openssl dgst -sha256 mensaje.txt
SHA2-256(mensaje.txt)= 6ae8c5e8c5286c171e09017e4ce1a7a4400ec13881b3e9124f91f7fbaea547f4
```

Compara ambos hashes: ¿Cuántos caracteres coinciden en la misma posición? ¿Qué propiedad demuestra esto?

Ningún carácter está en la misma posición; todo el hash ha cambiado. Esto demuestra que el mínimo cambio de un carácter cambia por completo el hash.

`echo` y `echo -n`

5. Genera el hash de la misma cadena con `echo` y con `echo -n`. ¿Por qué son diferentes?

Son diferentes porque `echo` añade un carácter de nueva línea al final de la cadena. En cambio, `echo -n` no lo añade.

Preguntas

a) ¿Es posible «descifrar» un hash?

No se puede descifrar porque el hash generado no se puede revertir, pero sí se pueden realizar ataques de fuerza bruta para intentar saber el texto original del archivo.

b) ¿Qué es una colisión y por qué MD5 y SHA-1 se consideran inseguros?

MD5 y SHA-1 se consideran inseguros porque se han encontrado métodos prácticos para generar colisiones:

- **MD5:** es vulnerable desde hace años; se pueden crear archivos distintos con el mismo hash.
- **SHA-1:** también tiene colisiones demostradas.

Apartado 2. Verificación de archivos

5. Verificación después de borrar `c.txt`

```
 sha256sum -c SUMAS.sha256
a.txt: La suma coincide
b.txt: La suma no coincide
sha256sum: c.txt: No existe el archivo o el directorio
c.txt: FAILED open or read
sha256sum: ATENCIÓN: no se pudo leer 1 fichero listado
```

 Captura de la verificación con c.txt eliminado

En el anterior apartado dice que el hash no coincide y ahora es que directamente no encuentra el archivo.

6. Opción `--ignore-missing`

Comprueba cómo se utilizaría la opción `--ignore-missing` y explica en qué situación real es útil.

Es útil cuando tienes varios archivos que quieres verificar y sus hashes han cambiado, para que no salgan los errores de los archivos que no están y no aparezcan tantas veces por terminal.

Apartado 3. Verificación de una descarga real

Preguntas

a) ¿Por qué no basta con comprobar el hash si lo has descargado del mismo servidor que el fichero?

Si un atacante logra comprometer el servidor de descargas, tiene la capacidad de modificar tanto la imagen `.iso` como el fichero de sumas (`SHA256SUMS`) para que ambos coincidan con la versión maliciosa introducida.

Al calcular el hash localmente y compararlo con el fichero de sumas descargado del mismo servidor alterado, el sistema dará por buena una ISO manipulada.

b) Diferencia entre integridad y autenticidad

Integridad: propiedad que asegura que los datos no han sido alterados, corrompidos o modificados, ni accidental ni maliciosamente, durante su almacenamiento o transmisión.

Autenticidad: propiedad que valida la identidad del creador o emisor del fichero, garantizando que proviene de una fuente legítima y confiable.

Comprobación del hash

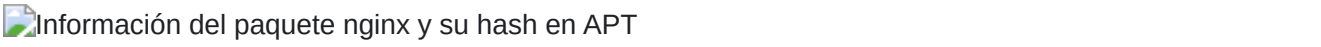
 Captura de la verificación de la descarga

Apartado 4. Verificación de paquetes

3. Descarga y comprobación del paquete

Descarga el paquete sin instalarlo con `apt download` y calcula su SHA-256. ¿Coincide con el que indica el repositorio?

Sí, coincide con la del repositorio.

 Información del paquete nginx y su hash en APT

 Cálculo del SHA-256 del paquete descargado

Apartado 4.2. Verificación de paquetes instalados

 Actualización de los índices de paquetes

Apartado 3

Esto indica que el archivo instalado difiere del original del paquete. La copia de seguridad conserva el binario correcto y permite restaurarlo.

 Detección de la modificación del paquete tree

 Verificación del paquete tree con debsums

 Verificación final del paquete tree

Preguntas

a) ¿Por qué usar `[trusted=yes]` en un repositorio es una mala práctica de seguridad?

Desactiva la verificación de autenticidad y firma del repositorio. Un atacante podría introducir paquetes maliciosos mediante un repositorio comprometido o un ataque *Man-in-the-Middle*, sin que APT los rechace.

b) ¿Qué limitación tiene `debsums` frente a un atacante que ya tiene acceso root?

No protege frente a un atacante con acceso `root`: puede modificar los ficheros instalados y también las bases de datos, sumas de comprobación o incluso el propio programa `debsums`. Por tanto, sus resultados podrían falsificarse o evitarse.

Apartado 5. Hashes de contraseñas en Linux

Información del usuario

```
username: alumno_hash
contraseña: $y$j9T$C6Lp2nDv2pgf41.3aqrCf1$EmdMqCeOcLCPvAIGEuAylHkezVwecMnSkv3rjKmOPRD
fecha del último cambio de contraseña: 20766
edad mínima de contraseña: 0
edad máxima de contraseña: 99999
periodo de aviso de caducidad: 7
```

 Contenido de /etc/shadow del usuario alumno_hash

```
$y$ --> algoritmo es el yescrypt
$j9T$ --> parámetros de configuración
$C6Lp2nDv2pgf41.3aqrCf1$ --> salt
$EmdMqCeOcLCPvAIGEuAylHkezVwecMnSkv3rjKmOPRD$ --> hash resultante
```

Hash de la misma contraseña

```
carlos@carlos-VirtualBox:~$ openssl passwd -6 Hash2026!
$6$sH4PgcUdR6G.K3Ro$VYOv5Z5HuY6BKammBTOa5BaHkA1Ks1yjhLRhu5xF2ubc3dEHfBy0W1K4tEXQU8L2SlBFCWyQpp1Kx0RrbG5TA.
carlos@carlos-VirtualBox:~$ openssl passwd -6 Hash2026!
$6$Oz6cpaAvlZl4KLgi$MReyJuK656bJyCYEIZomqsQDT.7anBXR6D7QF2eHQUJzxJXB211Mtp23k9BNG3yzOwjYBts2SSKnRbioqJIU8.
carlos@carlos-VirtualBox:~$
```

Los hashes nunca serán iguales porque cada vez se generará uno nuevo por seguridad, para que no haya dos hashes iguales.

 Generación de dos hashes para la misma contraseña

Apartado 4

```
carlos@carlos-VirtualBox:~$ openssl passwd -salt C6Lp2nDv2pgf41.3aqrCf1 -6 Hash2026!
$6$C6Lp2nDv2pgf41.3$ORzEZNzf1AjdKLsh74/wdTiOddFMCKnEEX/cD4IZqWwaUDupv2PG0F0iM89XMMlyyvykPKQlKTo9KQdP/D1mH.
```

Sí, obtiene la misma salt que el usuario `alumno_hash`.

Apartado 5

Solo los dos primeros parámetros son iguales; todo lo demás es diferente al primer usuario. Lo que no cambia son las fechas de mínimo y máximo y el cambio de contraseña.

Preguntas

a) ¿Por qué se usan algoritmos como yescrypt o bcrypt en lugar de SHA-256 para contraseñas?

Es una tabla de consultas precalculadas que asigna valores hash a sus entradas de texto plano originales.

b) ¿Qué es una *rainbow table* y cómo lo neutraliza la salt?

Una tabla *rainbow* es una tabla de consulta precalculada que asigna valores hash a sus entradas de texto plano originales, en lugar de calcular el hash de cada contraseña posible durante un ataque.

La defensa es el salteo: agregar un valor aleatorio a cada contraseña antes de aplicarle el hash.